

Python: str, list, set, dict, tuple + regex (re) — A4

Strings (str)		Tuples (tuple)		Sets (set)	
s.strip() / rstrip() / rstrip()	Trim whitespace/chars	t.count(x) / index(x)	Only tuple methods	s.add(x)	One element
s.lower() / upper() / title()	Case transforms	a, b, c = t	Unpack	s.update(iterable)	Many elements
s.split(",") / rsplit()	Split to list	a, *mid, z = t	Star unpack	s.remove(x) / discard(x)	Remove (error / safe)
",".join(parts)	List to string	(x,) single	One-element tuple	s.pop() / clear()	Arbitrary pop / empty
s.replace(old, new)	Replace substring	t1 + t2	Concatenate	a b	Union
s.startswith() / endswith()	Prefix/suffix check	x in t	Membership	a & b	Intersection
s.find() / rfind()	Index or -1	tuple(iterable)	Construct	a - b	Difference
s.count(sub)	Count occurrences	d[(region, sku)]	Composite dict key	a ^ b	Symmetric difference
s.isdigit() / isalpha() / isalnum()	Char-type checks	return a, b	Multi return	x in s	O(1) membership
s.partition(sep)	Head, sep, tail	sorted(t)	Sort → list	s.issubset(t) / issuperset()	Containment
s.removeprefix() / removesuffix()	Py3.9+ trim affix	hash(t) if no mutables	Hashable record	s.isdisjoint(t)	No overlap
s.zfill(n) / ljust() / rjust()	Pad / align			set(iterable)	Dedupe (unordered)
f"{x}" / format()	Interpolation			frozenset(s)	Immutable set
s[1:4] / s[::-1]	Slice / reverse			{x for x in it if cond}	Set comprehension

Lists (list)		Dictionaries (dict)		Regex (re module)	
lst.append(x)	Add end	d[key] = val	Assign	re.match(pat, s)	Start of string
lst.extend(iterable)	Add many	d.get(k, default)	Safe read	re.search(pat, s)	First anywhere
lst.insert(i, x)	Insert at i	d.setdefault(k, default)	Get or set	re.findall(pat, s)	All matches
lst.pop([i])	Remove & return	d.keys() / values() / items()	Views	re.sub(pat, repl, s)	Replace
lst.remove(x)	Remove first x	d.update(other)	Merge in-place	re.split(pat, s)	Split on pattern
lst.clear()	Empty list	d other / =	Merge Py3.9+	re.compile(pat)	Reuse pattern
lst.sort(key=fn, reverse=)	Sort in-place	d.pop(k) / popitem()	Remove pair	m.group() / group(1)	Whole / group
sorted(lst)	New sorted list	del d[k] / clear()	Delete / empty	m.groups()	All groups tuple
lst.reverse()	Reverse in-place	k in d	Key membership	r"\d+" raw	Raw string for \d
lst.index(x) / count(x)	Find / count	dict.fromkeys(keys, v)	Init keys	. ^ \$ \b	Any, start, end, word
lst.copy() / lst[:]	Shallow copy	{k: f(v) for k,v in d.items()}	Dict comp	\d \w \s \D \W \S	Digit, word, space
[x*2 for x in lst]	List comprehension	{r['id']: r for r in rows}	Lookup table	+ * ? {n,m}	Quantifiers
[x for x in lst if cond]	Filter comp	copy() / deepcopy()	Shallow / deep	[abc] [a-z] [^0-9]	Character class
list(map(fn, lst))	Transform			() (?:)	Group / alt / non-cap
list(filter(pred, lst))	Filter iterator			*? +?	Non-greedy

~80 ops • str, tuple, set | list, dict, regex • A4 • DE-focused